

CS2104 Report for Week 01

Week 01 Exercise Report

Sim Yue Yang (Student ID, yueyangsim02@gmail.com)

August 16, 2026

Abstract

This document contains the solutions to the Week 01 exercise. The code is provided in a separate file along with this document.

...a picture, maybe...

...a relevant quote, possibly...
– the author of the quote

Contents

1	Overall Introduction	4
2	Exercise 1: Self-reference	5
2.1	Statement	5
2.2	Introduction	5
2.3	Solution	6
2.4	Conclusion	6
3	Exercise 2: Media involving media	7
3.1	Statement	7
3.2	Introduction	7
3.3	Solution	7
3.4	Conclusion	7
4	Exercise 3: A multiple-choice question	8
4.1	Statement	8
4.2	Introduction	8
4.3	Solution	8
4.4	Conclusion	8
5	Exercise 4: The Sorcerer's Apprentice	9
5.1	Statement	9
5.2	Introduction	9
5.3	Solution	9
5.4	Conclusion	9
6	Exercise 5: Implementing the Sigma notation using a for loop	10
6.1	Statement	10
6.2	Introduction	11
6.3	Solution	11
6.4	Conclusion	11
7	Exercise 6: Sums of floor divisions	12
7.1	Statement	12
7.2	Introduction	12
7.3	Solution	12
7.4	Conclusion	12
8	Exercise 7: Nested sums and binomials	13
8.1	Statement	13
8.2	Introduction	14
8.3	Solution	14

8.4	Conclusion	15
9	Exercise 8: Catalan numbers	17
9.1	Statement	17
9.2	Introduction	17
9.3	Solution	17
9.4	Conclusion	18
10	Exercise 9: Fibonacci sequence	19
10.1	Statement	19
10.2	Introduction	19
10.3	Solution	19
10.4	Conclusion	20
11	Exercise 10: Factorials (decreasing bounds)	21
11.1	Statement	21
11.2	Introduction	21
11.3	Solution	21
11.4	Conclusion	22
12	Exercise 11: Factorials (increasing bounds)	23
12.1	Statement	23
12.2	Introduction	23
12.3	Solution	23
12.4	Conclusion	24
13	Food for Thought	25
13.1	Statement	25
13.2	Solution	25
14	Overall Conclusion	26

1 Overall Introduction

- This document is the document that documents the solutions for the questions in the Week01 handin document.
- This document documents the solutions for the questions in the Week01 handin.
- This point is situated in week 01 of the semester.
- Title, abstract, content table, introduction, exercises (question, introduction, solution, conclusion for each exercise), and an overall conclusion.

2 Exercise 1: Self-reference

2.1 Statement

The first lecture started with the DSB jingles #1 and #2 at toglyde.dk.

Using the fixed-do solfège, this three-note jingle D/Eb/Bb (pronounced D/S/B in Danish, i.e., D Sharp B) is the acronym of DSB, the Danish Railroad Company “Danske Statsbaner”, and is played in Danish rail stations when a train arrives. It is due to the Danish composer Niels Viggo Bentzon.

This jingle illustrates self-reference since reading aloud yields the name DSB.

Other examples of self-reference include the liar paradox and René Magritte’s painting “The Treachery of Images”.

1.a. Identify the self-reference in the cover of blues guitarist Coco Montoya’s album “The Writing on the Wall”.

1.b. Identify the self-reference in the cover of Gail Carson Levine’s book “Writing Magic: Creating Stories that Fly”.

1.c. Identify the self-reference in Alan Perlis’s programming epigram #59:

In English, every word can be verbed.

All the self-references above are direct. Some self-references are indirect in that something named A refers to something named B that refers to A. For example, for any natural number n , the number $n+1$ is even if the number n is odd, and the number $n+1$ is odd if the number n is even.

1.d. Identify the indirect self-reference in the following story:

Two women walk into a bar and talk about the Bechdel-Wallace test.

1.e. Identify the self-reference, if there is one, in the following sentences:

- **1.e.1.** This sentence no verb.
- **1.e.2.** Nostalgia isn’t what it used to be.
- **1.e.3.** Fashion is what goes out of fashion. — Coco Chanel
- **1.e.4.** To be or not to be. (phrase · 1942 film)

2.2 Introduction

- The point of the exercise is to identify self-references across various media and sentences.
- The exercise is solved by inspecting each example and describing how it refers to itself.

2.3 Solution

1.a. The album cover features a literal writing on the wall.

1.b. The person on the cover is writing the word “Magic”, taking the phrase “Writing Magic” literally.

1.c. The word “verb” is used as a verb in the sentence.

1.d. The Bechdel-Wallace test checks if 2 female characters in a story converse about something other than a male character. This story about two women conversing about the Bechdel-Wallace test itself passes the Bechdel-Wallace test.

1.e.1 None.

1.e.2 The subject is nostalgic for the concept of nostalgia.

1.e.3 Fashion is ever-changing, so all fashion can eventually become irrelevant (“goes out of fashion”).

1.e.4 None.

2.4 Conclusion

- The point of the exercise was to practice spotting self-reference.
- The learning points illustrated were direct self-reference, and indirect self-reference where A refers to B which refers back to A.

3 Exercise 2: Media involving media

3.1 Statement

Can you think of

- a book involving a book,
- a theater play involving a theater play, or
- a movie involving a movie?

More generally, can you think of one of these mediums (book, theater play, movie, TV series, etc.) involving one of these mediums?

If so, briefly describe a couple of them and their involvement.

Subsidiary question: Give an example where self-reference is used in Computer Science.

Partial solution for Exercise 2. “A Midsummer Night’s Dream” qualifies because in Scene 1 of Act V, Pyramus and Thisbe is a play within a play.

3.2 Introduction

- The point of the exercise is to find a medium that contains another instance of a medium, and an example of self-reference in Computer Science.
- The exercise is solved by recalling examples from memory.

3.3 Solution

- In Ridge Racer 7, the playable mini-game that appears during loading screens is Galaxian, the classic Namco arcade shooter.
- In computer science, recursive functions are functions that call themselves as part of their execution.

3.4 Conclusion

- The point of the exercise was to spot self-reference in nested media and Computer Science.
- The exercise was solved by giving real-life examples.
- The learning point illustrated was that recursion is a form of self-reference.

4 Exercise 3: A multiple-choice question

4.1 Statement

Facetiously, you are asked whether there exists a word in English that contains all of the letters “a”, “e”, “i”, “o”, “u”, and “y”, in that order. Does there exist such a word in English?

Possible answers:

- **a.** Yes, unquestionably. Constant vigilance!
- **b.** No.
- **c.** I cannot possibly know this, and furthermore this facetious question is nefariously out of scope in CS2104.

4.2 Introduction

- The point of the exercise is to determine whether an English word contains the letters a, e, i, o, u, y in that order.
- The exercise is solved by finding one such word.

4.3 Solution

- a. “Adventitiously”.

4.4 Conclusion

- The point of the exercise was to copy-paste the question into google.
- The exercise was solved by finding the word ”adventitiously”.
- The learning point illustrated was that the word ”adventitiously” exists.

5 Exercise 4: The Sorcerer's Apprentice

5.1 Statement

In reference to LLMs, please view the short piece “The Sorcerer’s Apprentice” in “Fantasia”.

N.B.: While you are at it, expand your musical culture with Paul Dukas’s score, as a continuation of Johann Strauss II’s waltz that you so magnificently rendered at the beginning of the first lecture (The Blue Danube).

5.2 Introduction

- The point of the exercise is to relate “The Sorcerer’s Apprentice” to LLMs.
- The exercise is solved by reading the short piece.

5.3 Solution

- The story is about the apprentice using magic, losing control of it, which results in a mess. If we view this as analogy where the apprentice is a CS undergrad and the magic is the LLM, we could say that a CS undergrad using LLMs wantonly will get nowhere.

5.4 Conclusion

- The point of the exercise was to draw an analogy between LLMs and magic.
- The exercise was solved by reading the piece.
- The learning point illustrated was the risk of using LLMs without knowledge over its outputs.

6 Exercise 5: Implementing the Sigma notation using a for loop

6.1 Statement

See Capital-sigma notation.

The Sigma notation reads, given a non-negative integer n and a function f from (non-negative) integers to integers,

`sum for i = 0 to n of f(i)`

and it stands for

$f(0) + f(1) + \dots + f(n)$

In this notation, 0 is the lower bound, n is the upper bound, and $f(i)$ is the body.

Here, the notation is extended to negative integers in that given a negative integer m and a function f ,

`sum for i = 0 to m of f(i)`

stands for

0

The starting point of this exercise is to implement — in the programming language of your choice — a routine / function / procedure / method / what-have-you named **Sigma0** that takes two parameters: an integer n and a function from integer to integer f . Applying **Sigma0** to n and f should yield the result of $f(0) + f(1) + \dots + f(n)$ when n is non-negative, and 0 otherwise.

5.a. Devise meaningful tests for your implementation. For example, f could be a constant function, or the identity function, or the function that maps an integer i to the i +first odd natural number (namely $i.2*i + 1$ using the lambda notation; then what is computed is $1 + 3 + 5 + \dots$).

5.b. Implement **Sigma0** using a for loop.

5.c. Check that your implementation passes your unit tests. (The testing should be automatic, and since what goes well should go without saying, it should only report the tests that failed.)

5.d. Implement a function named **Sigma1** that takes two parameters: an integer n and a function from integer to integer f . Applying **Sigma1** to n and f should yield the result of $f(1) + \dots + f(n)$ when n is positive, and 0 otherwise.

6.2 Introduction

- The point of the exercise is to implement the Sigma notation with a for loop and test it.
- The exercise is solved by accumulating $f(i)$ over the range in a loop, returning 0 for a negative upper bound.

6.3 Solution

See functions Sigma0 and Sigma1 in week01.cpp.

6.4 Conclusion

- The point of the exercise was to implement Sigma notation as code and verify it.
- The exercise was solved with a for-loop accumulator guarding negative bounds, checked by automated tests.
- The learning points illustrated were summation as iteration, higher-order functions, and automated unit testing.

7 Exercise 6: Sums of floor divisions

7.1 Statement

6.a. Given a non-negative integer n , what is the result of

`sum for i = 0 to 2 * n of floor(n/2)`

where $\text{floor}(5/2) = 2$ and $\text{floor}(6/2) = 3$.

6.b. Given a non-negative integer n , what is the result of

`sum for i = 0 to 3 * n of floor(n/3)`

6.c. Given a non-negative integer n , what is the result of

`sum for i = 0 to 4 * n of floor(n/4)`

Hint: Use the Online Encyclopedia of Integer Sequences.

6.d. Given a non-negative integer n , what is the result of

`sum for i = 0 to 1 * n of floor(n/1)`

(Can you simplify this formulation?)

7.2 Introduction

- The point of the exercise is to find closed forms for sums whose body is a constant floor term.
- The exercise is solved by noting the body does not depend on i , so the result is the body times the number of terms.

7.3 Solution

For each part let r be the result.

6.a. $r = (2n + 1) \left\lfloor \frac{n}{2} \right\rfloor$

6.b. $r = (3n + 1) \left\lfloor \frac{n}{3} \right\rfloor$

6.c. $r = (4n + 1) \left\lfloor \frac{n}{4} \right\rfloor$

6.d. $r = (n + 1)(n)$

7.4 Conclusion

- The point of the exercise was to derive closed forms for constant-body sums.
- The exercise was solved by multiplying the number of terms by the constant body.
- The learning point illustrated was that a constant summand gives $(\text{count}) \times (\text{value})$.

8 Exercise 7: Nested sums and binomials

8.1 Statement

7.a. What is the result of

1

7.b. Given a non-negative integer x , what is the result of

sum for $i_1 = 0$ to x of
1

7.c. Given a non-negative integer x , what is the result of

sum for $i_2 = 0$ to x of
sum for $i_1 = 0$ to i_2 of
1

7.d. Given a non-negative integer x , what is the result of

sum for $i_3 = 0$ to x of
sum for $i_2 = 0$ to i_3 of
sum for $i_1 = 0$ to i_2 of
1

7.e. Given a non-negative integer x , what is the result of

sum for $i_4 = 0$ to x of
sum for $i_3 = 0$ to i_4 of
sum for $i_2 = 0$ to i_3 of
sum for $i_1 = 0$ to i_2 of
1

7.f. Implement a function that maps two non-negative integers N and x to

sum for $i_N = 0$ to x of
sum for $i_{N-1} = 0$ to i_N of
...
sum for $i_2 = 0$ to i_3 of
sum for $i_1 = 0$ to i_2 of
1

$$\sum_{i_N=0}^x \sum_{i_{N-1}=0}^{i_N} \cdots \sum_{i_2=0}^{i_3} \sum_{i_1=0}^{i_2} 1$$

and empirically characterize its result. If possible (ha, ha, ha), justify your characterization.

N.B.: Since there is no ambiguity, feel free to write

```
sum for i = 0 to x of
sum for i = 0 to i of
...
sum for i = 0 to i of
sum for i = 0 to i of
1
```

but then you need to say that there are N occurrences of “sum” since this information is no longer present in the notation.

8.2 Introduction

- The point of the exercise is to characterize the result of N nested sums of 1.
- The exercise is solved by computing values into a sequence, and looking them up on OEIS.

8.3 Solution

For each part let r be the result.

7.a. $r = 1$

7.b. $r = x + 1$

7.c. $r = \frac{(x+1)(x+2)}{2}$

7.d. $r = \frac{(x+1)(x+2)(x+3)}{6}$

7.e. $r = \frac{(x+1)(x+2)(x+3)(x+4)}{24}$

7.f.

See function q7 in week01.cpp.

The following are the outputs for the function for $N \in [0, 5]$ and $x \in [0, 5]$:

```
N = 0, x = 0 | 1
N = 0, x = 1 | 1
N = 0, x = 2 | 1
N = 0, x = 3 | 1
N = 0, x = 4 | 1
N = 0, x = 5 | 1
```

N = 1, x = 0 | 1
 N = 1, x = 1 | 2
 N = 1, x = 2 | 3
 N = 1, x = 3 | 4
 N = 1, x = 4 | 5
 N = 1, x = 5 | 6
 N = 2, x = 0 | 1
 N = 2, x = 1 | 3
 N = 2, x = 2 | 6
 N = 2, x = 3 | 10
 N = 2, x = 4 | 15
 N = 2, x = 5 | 21
 N = 3, x = 0 | 1
 N = 3, x = 1 | 4
 N = 3, x = 2 | 10
 N = 3, x = 3 | 20
 N = 3, x = 4 | 35
 N = 3, x = 5 | 56
 N = 4, x = 0 | 1
 N = 4, x = 1 | 5
 N = 4, x = 2 | 15
 N = 4, x = 3 | 35
 N = 4, x = 4 | 70
 N = 4, x = 5 | 126
 N = 5, x = 0 | 1
 N = 5, x = 1 | 6
 N = 5, x = 2 | 21
 N = 5, x = 3 | 56
 N = 5, x = 4 | 126
 N = 5, x = 5 | 252

I have manually calculated the results and verified them to be accurate.

By searching the OEIS, these seem to be the binomial results of $\binom{x+N}{N}$.

It can be justified by observing that $\binom{a}{b} = \frac{a \cdot (a-1) \cdot \dots \cdot (a-b+1)}{k \cdot (k-1) \cdot \dots \cdot 1}$, so all that's needed is to offset the x by $+N$ in the top part.

8.4 Conclusion

- The point of the exercise was to identify the value of N nested sums of 1.
- The exercise was solved by recognizing outputs as $\binom{x+N}{N}$.

- The learning point illustrated was that nested sums count combinations.

9 Exercise 8: Catalan numbers

9.1 Statement

Implement a function that maps non-negative integer N to

```
sum for iN = 0 to 0 of
sum for iN-1 = 0 to iN+1 of
...
sum for i2 = 0 to i3+1 of
sum for i1 = 0 to i2+1 of
1
```

$$\sum_{i_N=0}^0 \sum_{i_{N-1}=0}^{i_N+1} \cdots \sum_{i_2=0}^{i_3+1} \sum_{i_1=0}^{i_2+1} 1$$

and empirically characterize its result. If possible (ha, ha, ha), justify your characterization.

9.2 Introduction

- The point of the exercise is to characterize a nested sum whose bounds are shifted by $+1$.
- The exercise is solved by a recursive implementation, tabulating the outputs, and matching against the OEIS.

9.3 Solution

See function `q8` in `week01.cpp`.

The following are the outputs for the function for $N \in [0, 10]$:

```
N = 0 | 1
N = 1 | 1
N = 2 | 2
N = 3 | 5
N = 4 | 14
N = 5 | 42
N = 6 | 132
N = 7 | 429
N = 8 | 1430
N = 9 | 4862
N = 10 | 16796
```

I have manually calculated the results and verified them to be accurate.

By searching the OEIS, these seem to be the Catalan sequence.

After reading the Wikipedia page on Catalan numbers, I have concluded that I do not understand it, and therefore will not attempt to justify the characterization.

9.4 Conclusion

- To identify the value of the shifted nested sum.
- By recursion and an OEIS lookup, giving the Catalan numbers.
- The Catalan sequence.

10 Exercise 9: Fibonacci sequence

10.1 Statement

Implement a function that maps non-negative integer N

```
sum for iN = 0 to 0 of
sum for iN-1 = 0 to 1-iN of
...
sum for i2 = 0 to 1-i3 of
sum for i1 = 0 to 1-i2 of
1
```

$$\sum_{i_N=0}^0 \sum_{i_{N-1}=0}^{1-i_N} \cdots \sum_{i_2=0}^{1-i_3} \sum_{i_1=0}^{1-i_2} 1$$

and characterize its result. (Remember that if the upper bound of a summation is negative, the result is 0.) If possible (ha, ha, ha), justify your characterization.

10.2 Introduction

- To characterize a nested sum whose bounds are of the form $1 - i$.
- The exercise is solved by a recursive implementation, tabulating the outputs, and matching against the OEIS.

10.3 Solution

See function q9 in week01.cpp.

The following are the outputs for the function for $N \in [0, 10]$:

```
N = 0 | 1
N = 1 | 1
N = 2 | 2
N = 3 | 3
N = 4 | 5
N = 5 | 8
N = 6 | 13
N = 7 | 21
N = 8 | 34
N = 9 | 55
N = 10 | 89
```

I have manually calculated the results and verified them to be accurate.

By searching the OEIS, these seem to be the N -th number of the fibonacci sequence.

It can be justified by observing that to calculate for N , the count of the 2nd sigma operation is between 0 and 1. When that count is 0, it recursively calculates for $N - 1$, and when the count is 1, it recursively calculates for $N - 2$. Then the 2nd sigma operation simply sums up the result.

10.4 Conclusion

- To identify the value of this nested sum.
- By recursion producing $f(N - 1) + f(N - 2)$, giving the Fibonacci numbers.
- The Fibonacci recurrence arising from the summation bounds.

11 Exercise 10: Factorials (decreasing bounds)

11.1 Statement

Implement a function that maps non-negative integer N

```
sum for i = 0 to N of
sum for i = 0 to N-1 of
...
sum for i = 0 to 2 of
sum for i = 0 to 1 of
sum for i = 0 to 0 of
1
```

$$\sum_{i=0}^N \sum_{i=0}^{N-1} \cdots \sum_{i=0}^2 \sum_{i=0}^1 \sum_{i=0}^0 1$$

and characterize its result. Justify your characterization.

11.2 Introduction

- To characterize nested sums with constant bounds decreasing from N to 0.
- By multiplying the layer sizes, tabulating the outputs, and matching against the OEIS.

11.3 Solution

See function `q10` in `week01.cpp`.

The following are the outputs for the function for $N \in [0, 10]$:

```
N = 0 | 1
N = 1 | 2
N = 2 | 6
N = 3 | 24
N = 4 | 120
N = 5 | 720
N = 6 | 5040
N = 7 | 40320
N = 8 | 362880
N = 9 | 3628800
N = 10 | 39916800
```

I have manually calculated the results and verified them to be accurate.
By searching the OEIS, these seem to be the $N + 1$ factorials.

It can be justified by observing that each Sigma operation nests another Sigma operation within that decreases the upper bound by 1, from N to 0. Since it's all just adding up 1s, the final sum can be calculated by finding the number of total rounds by all Sigma operations. And the multiplied terms are just the factorial operation $(N + 1)!$ in reverse.

11.4 Conclusion

- To identify the value of these nested sums.
- By taking the product of the layer sizes, giving $(N + 1)!$.
- Factorial as a product of counts.

12 Exercise 11: Factorials (increasing bounds)

12.1 Statement

Implement a function that maps non-negative integer N

```
sum for i = 0 to 0 of
sum for i = 0 to 1 of
...
sum for i = 0 to N-1 of
sum for i = 0 to N of
1
```

$$\sum_{i=0}^0 \sum_{i=0}^1 \cdots \sum_{i=0}^{N-1} \sum_{i=0}^N 1$$

and characterize its result. Justify your characterization.

12.2 Introduction

- To characterize nested sums with constant bounds increasing from 0 to N .
- By multiplying the layer sizes, tabulating the outputs, and matching against the OEIS.

12.3 Solution

See function `q11` in `week01.cpp`.

The following are the outputs for the function for $N \in [0, 10]$:

```
N = 0 | 1
N = 1 | 2
N = 2 | 6
N = 3 | 24
N = 4 | 120
N = 5 | 720
N = 6 | 5040
N = 7 | 40320
N = 8 | 362880
N = 9 | 3628800
N = 10 | 39916800
```

I have manually calculated the results and verified them to be accurate.
By searching the OEIS, these seem to be the $N + 1$ factorials.

It can be justified by observing that each Sigma operation nests another Sigma operation within that increases the upper bound by 1, from 0 up to N . Since it's all just adding up 1s, the final sum can be calculated by finding the number of total rounds by all Sigma operations. And the multiplied terms are just the factorial operation $(N + 1)!$ in reverse.

12.4 Conclusion

- To identify the value of these nested sums.
- By taking the product of the layer sizes, giving $(N + 1)!$ as in Exercise 10.
- The order of the bounds does not change the product.

13 Food for Thought

13.1 Statement

These nested sums are not efficient. (Why?) How could their efficiency be improved?

13.2 Solution

These nested sums, for a certain $N > 0$ requires the $N - 1$ levels of the nested sum to be computed before it can continue. The efficiency can be improved by finding a formula that allows for direct computation.

14 Overall Conclusion

The overall conclusion goes here:

- Where are we in the semester and what is the general topic addressed in this report?
- Which learning does this report document?